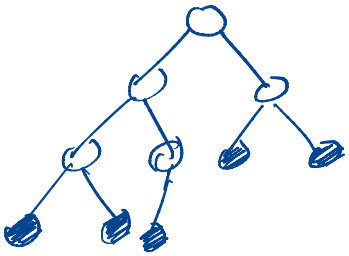
Heaps

Heaps: A (binary) heap data structure is an array-based object that can be viewed as a complete binary tree.

→ A binary tree that is filled at all levels except possibly the lowest which is filled from left



- An array $A[1:n]$ represents a heap object
  - → $A.heap\text{-}size$ represents number of elements in the heap.

$$0 \leq A.heap\text{-}size \leq n$$

and $A[1:A.heap\text{-}size]$ contains valid elements

- The tree is located at $A[1]$
- If $A.heap\text{-}size = 0$, then the heap is empty
- Given an index i of a node

$$Parent(i) = \lfloor i/2 \rfloor$$

$$left(i) = 2i$$

$$right(i) = 2i + 1$$

- Two kind of heaps:
  - max-heaps, and
  - min-heaps.

Both satisfy a heap-property.

- In a max-heap, the max-heap property is satisfied at every node i (other than root)

$$A[\text{parent}(i)] \geq A[i]$$

⇒ The largest element in a max-heap is stored at the root.

- A min-heap is organized in the opposite fashion: min-heap property

$A[\text{parent}(i)] \leq A[i]$, for every node i other than the root

⇒ Smallest element is stored at the root.

- Heap sort uses max-heaps
- Priority queues uses heaps.

- Height of a node in a heap is defined similar to trees as:

  # of edges on the longest simple downward path from node to leaf.

- Height of heap = height of root

- Since a heap of n-elements is based on a complete binary tree, its height is $\Theta(\lg n)$

  ⇒ Basic operations on heaps run in time proportional to $O(\lg n)$

ex.

Max-Heapify — $O(\lg n)$

maintains max-heap property

Build-Max-Heap — $O(n)$

create max-heap from unordered input

Heap-Sort — $O(n \lg n)$

Sort an array in-place

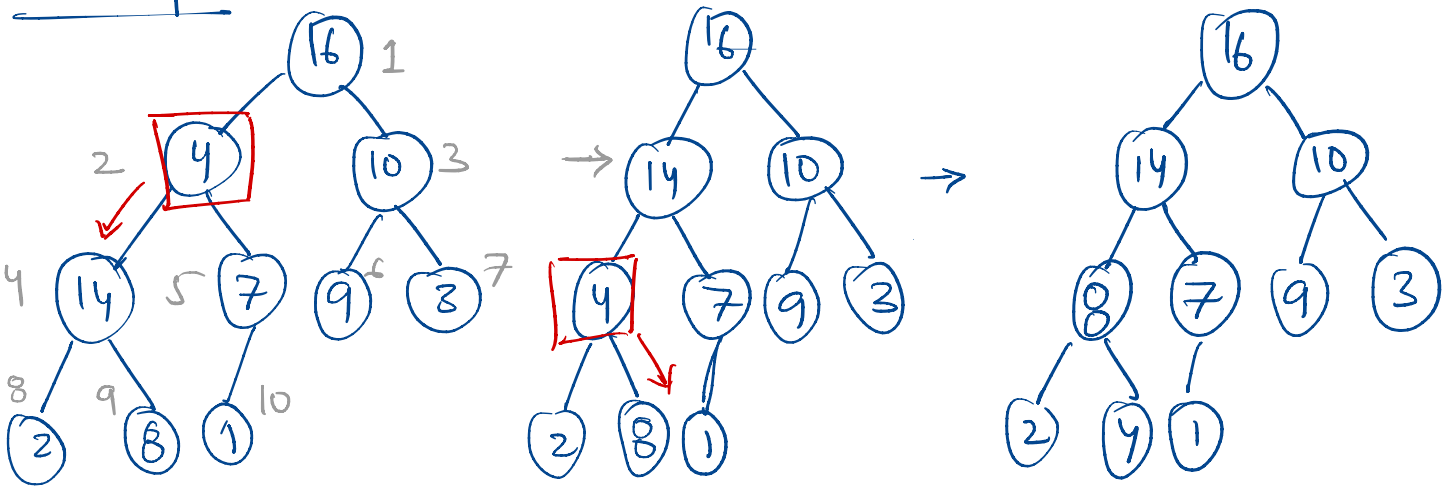
Max-Heap-Insert
~~Max-Heap-Extract-Max~~
Max-Heap-Insert-key
Max-Heap-Maximum
} run in $O(\lg n)$

- Maintaining heap property

Max-Heapify (A, i) : Assumes subtrees left(i) and right(i) are max-heaps, but $A[i]$ might be smaller than its children, thus violating the max-heap property

- Max-Heapify lets the value $A[i]$ "flow down" in the max-heap so that the subtree rooted at i obeys the max-heap property

Example:



Max-Heapify (A, i)

```
l = left(i)
r = right(i)
if l ≤ A.heap-size & A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size & A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    Max-Heapify(A, largest)
```

$O(\lg n)$

→ check max of A[i], A[left(i)] & A[right(i)] & swap with A[i]

→ Call Max-Heapify recursively on the subtree whose root was swapped.

Complexity analysis: For a tree rooted at i,

$\Theta(1)$: fix relationship amongst

$A[i]$, $A[\text{left}(i)]$, $A[\text{right}(i)]$

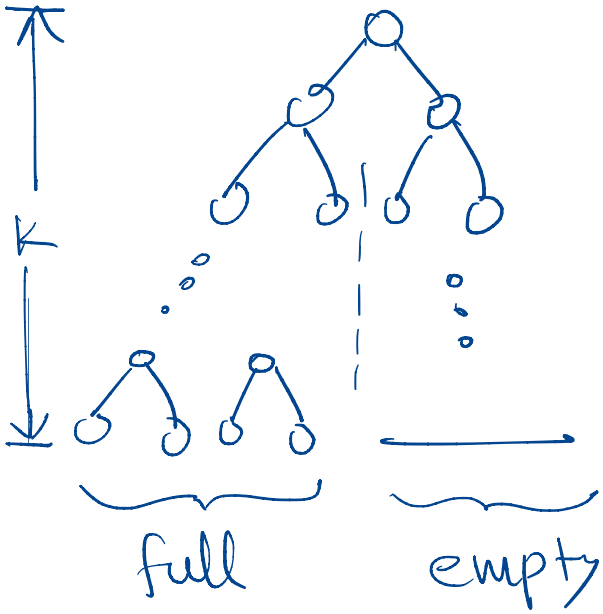
+

time to run Max-Heapify on a child subtree

fact: children's subtrees each have size at most $2n/3$

Proof: Extreme case for a heap is:

k-levels where last level is exactly half full



n_{left} = total # of nodes in the left subtree.

$$n_{left} = 2^{k-1} - 1$$

$$n_{total} = n_{left} + n_{right} + 1$$

$$\Rightarrow n_{total} = (2^{k-1} - 1) + (2^{k-2} - 1) + 1$$

$$\Rightarrow n_{total} = \left(2^{k-1} - 1\right) + \left(\frac{2^{k-1}}{2} - 1\right) + 1$$

$$= \left(2^{k-1} - 1\right) + \left(\frac{2^{k-1} - 1 - 1 + 2}{2}\right)$$

$$= \left(2^{k-1} - 1\right) + \left(\frac{\left(2^{k-1} - 1\right) + 1}{2}\right)$$

$$= n_{left} + \frac{n_{left} + 1}{2}$$

$$= \frac{3 n_{left} + 1}{2}$$

$$\Rightarrow n_{left} = \left(\frac{2 n_{total} - 1}{3}\right)$$

$$\Rightarrow \boxed{n_{left} < \frac{2 n_{total}}{3}} \quad \text{in general}$$

Therefore, time complexity of Max-Heapify is:

$$T(n) \leq T\left(\frac{2n}{3}\right) + \Theta(1)$$

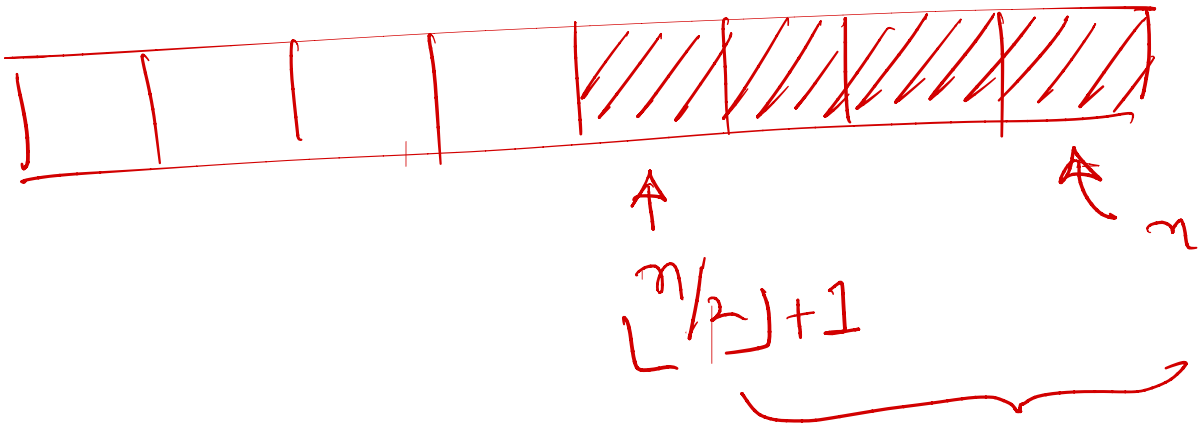
Use Master theorem (case 2)

$$\boxed{T(n) = O(\lg(n))}$$

- Building a heap

Build-~~Max-Heap~~ converts an array $A[1:n]$ into a max-heap by calling Max-Heapify in a bottom up manner.

Note: subarray $A[\lfloor n/2 \rfloor + 1 : n]$ are all leaves

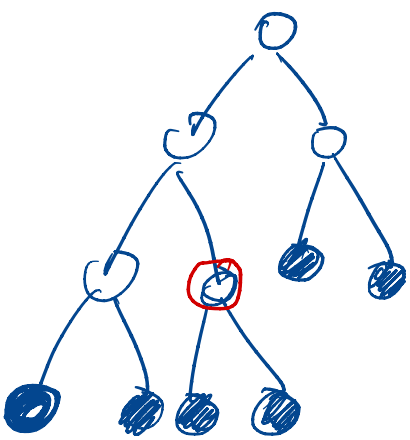


Build-Max-Heap(A, n)

A.heap-size = n

for i = ⌊n/2⌋ down to 1

    Max-Heapify(A, i)



O(n)

loop invariant : At the start of each iteration of the for loop, each node $(i+1), (i+2), \ldots, n$ is the root of a max-heap.

Complexity :

Note : ① An n-element heap has height $\lfloor \lg n \rfloor$

② For a given height h, there are at most $\lceil n/2^{(h+1)} \rceil$ nodes in an n-element heap

Cost of max-heapify on a node of height h is $O(h)$.

Upper bound of cost of Build-Max-Heap $= \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{(h+1)}} \right\rceil O(h)$

$= O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} h/2^h \right)$

Note ③ $\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = 2$

⇒ Running time of Build-Max-Heap :

$$O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right) = O\left(n \sum_{h=0}^{\infty} h/2^h \right) = O(n)$$

Heap Sort

Given an array $A[1:n]$

1) Call Build-Max-Heap to build a max-heap
2) Move the max element to the end by exchanging $A[1]$ with $A[n]$
3) Reduce heap size by 1
4) Call max-heapify at root to fix.

HeapSort (A, n)

Build-Max-Heap (A, n) → $O(n)$

for $i = n$ down to 2

  exchange $A[1]$ with $A[i]$

  A.heap-size = A.heap-size − 1

  Max-Heapify (A, 1)

$O(n \lg n)$

Loop invariant: At the start of the for loop the subarray $A[1:i]$ is max-heap containing i smallest elements of $A[1:n]$, the subarray $A[i+1:n]$ contains $(n-i)$ largest elements of $A[1:n]$, sorted.

Complexity of Heapsort : $T(n) = O(n) + (n-1)(\lg n)$

$= O(n \lg n)$

→ loose bound!

$$T(n) = O(n) + [\lg(n-1) + \lg(n-2) + \lg(n-3) + \cdots \lg(2)]$$

$$= O(n) + \sum_{i=n-1}^{2} \lg(i)$$

$$= O(n) + \lg\left((n-1)(n-2)\cdot(n-3)\cdots(2)\right)$$

$$= O(n) + \lg\left((n-1)!\right)$$

Recall Sterling's approx. $n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$

$$\Rightarrow T(n) = O(n) + O\left(\lg\left((n-1)^{n-1}\right)\right)$$

$$= O(n \lg n)$$

Priority Queue: A data structure for maintaining a set S of elements, each with an associated value (priority) called a key

- Max priority queue used max-heap
  - ↳ supports operations:
    - Insert (S, x, k): inset element x with key k into set S
      equiv. to $S = S \cup \{x\}$
    - Maximum (S) returns element with largest key
    - Extract-max(S) removes & returns element with largest key
    - Increase-key (S, x, k) increases the value of x's key to new value k (assuming $k \geq x.key$)

Max-Heap-Maximum(A)

```
if A.heap-size < 1
    | error "heap underflow"
return A[1]
```

$\Theta(1)$

Max-Heap-Extract-Maximum(A)

```
max = Max-Heap-Maximum(A)
A[1] = A[A.heap-size]
A.heap-size = A.heap-size - 1
Max-Heapify(A, 1)
return max
```

$O(\lg n)$

Heap-Increase-key (A, i, key)

if key < A[i]
    error "new key is smaller than current key"

A[i] = key → increase the key

while i > 1 and A[Parent(i)] < A[i]
    exchange A[i] with A[Parent(i)]
    i = Parent(i)

O(lg n)

i = Parent(i) → move up to fix upstream nodes

exchange A[i] with A[Parent(i)] → restore heap property at Parent

Insert:
- Takes key of new element as input
- expands the max-heap by addy a leaf node with key = -∞
- Call heap-increase-key to set the correct key & restore max-heap prop.

Max-Heap-Insert (A, key)

A.heap-size = A.heap-size + 1
A[A.heap-size] = -∞
Heap-Increase-key (A, A.heap-size, key)

O(lg n)